

Graph and Its Applications

Graph and Their Applications

Graph

- Nonlinear data Structures.
- A graph G consists of two properties:
 - (a) A set V of elements called vertices or nodes.
 - (b) A set E of connectors called edges such that each edge e is identified as $e = [u,v]$ (unordered pair of vertices). Here u and v are called endpoints of e and they are said to be the adjacent nodes or neighbors .
- The order of a graph is $|V|$ (the number of vertices).
- A graph's size is $|E|$, the number of edges.
- A tree is a graph with no cycle.

Example:

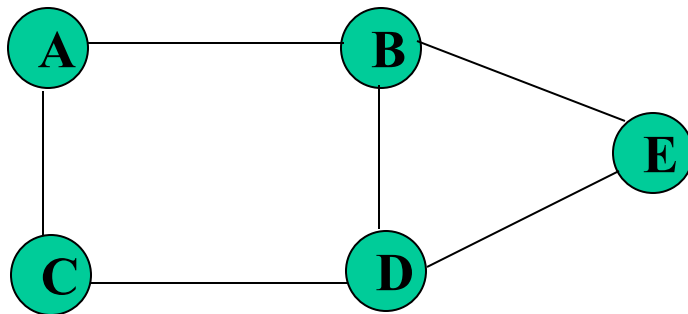
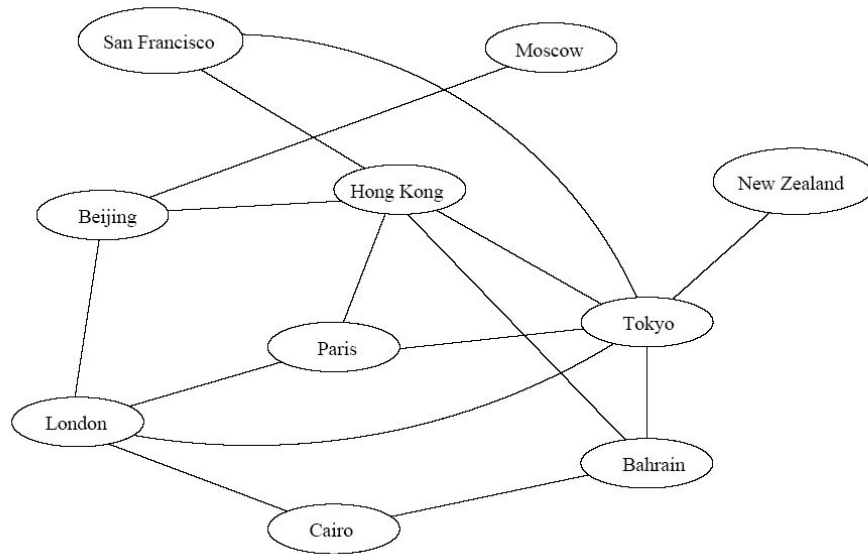


Figure1: A graph G_1

In Graph G_1

- 5 Vertices: $\{A, B, C, D, E\}$
- 6 Edges: $\{[A,B], [A,C], [B,D], [B,E], [C,D], [D,E]\}$

Application



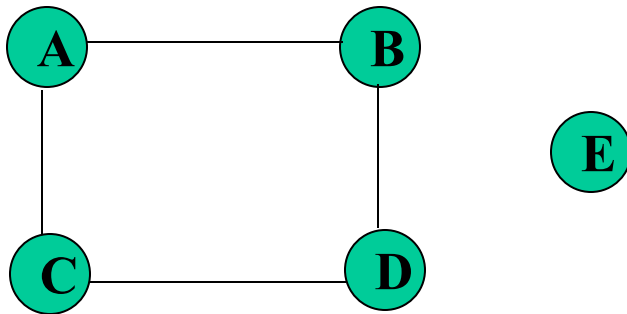
**Air flight
system**

- Each vertex represents a city
- Each edge represents a direct flight between two cities
- A query on **direct flights** = a query on whether an edge exists
- A query on **how to get to a location** = does a **path** exist from A to B
- We can even associate costs to **edges** (**weighted graphs**), then ask “what is the cheapest path from A to B”

Definitions

- **Isolated Node:** A vertex u with no edges.
- **Path:** A path P of length n from a node u to node v is defined as a sequence $n+1$ nodes such that $P=(v_0, v_1, \dots, v_n)$.
 - (a) Simple Path: if all nodes in path P are distinct.
 - (b) Closed Path: If node $v_0 = v_n$.
- **Cycle:** A closed path, with no other repeated vertices other than the starting and the ending vertices.

Example:



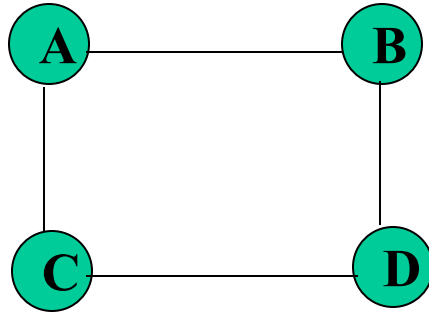
In graph G_2

- Isolate Node: E
- Path P : A, B, D, C
- Cycle C_1 : A, B, D, C, A

Figure2: Graph G_2

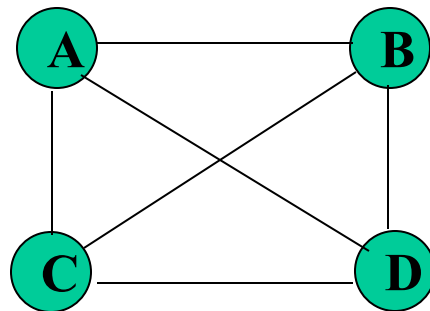
- **Connected Graph:** A graph is called connected if there is a simple path between any two of its nodes.

Example:



- **Complete Graph:** A graph G is complete if every node in graph G is adjacent to every other nodes. A complete graph with n nodes will have $n(n-1)/2$ edges.

Example:



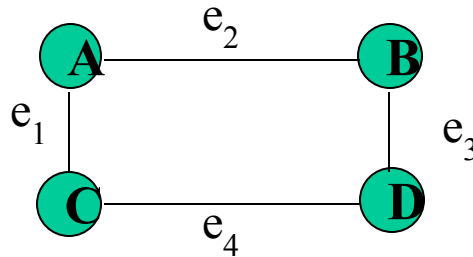
- **Tree Graph:** A connected graph with no cycle. If a tree graph has m nodes, then there are $m-1$ edges.

Example:



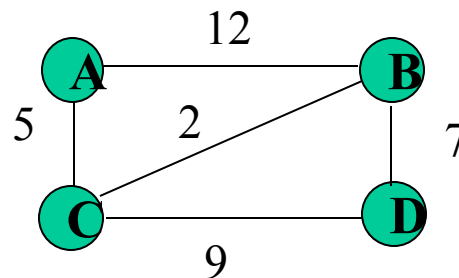
- **Labeled Graph:** A graph G is said to be labeled if its edges are assigned data.

Example:



- **Weighted Graph:** A labeled graph where each edge is assigned a nonnegative numerical value $w(e)$.

Example:

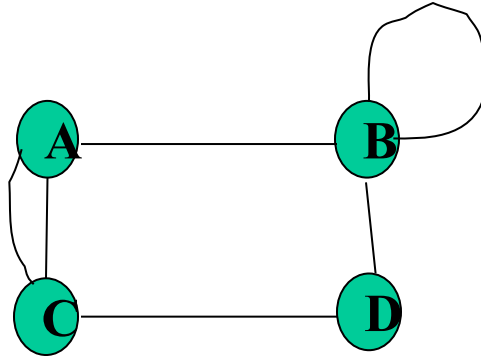


- **Multigraph:** A multigraph has the following properties:

(a) Mutiple Edges

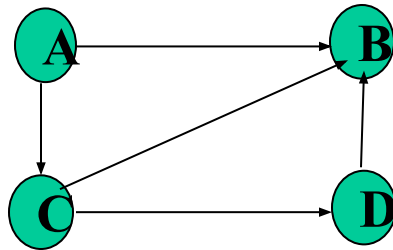
(b) Loops

Example:



- **Directed Graph (Digraph):** Each edge in graph has a direction such that $e = (u,v)$, ie. e begins at u and ends at v .

Example:



• **Degree of a Node:** No. of edges connected to a node.

(a) **Indegree:** No. of edges ending at a node.

(b) **Outdegree:** No. of edges beginning at a node.

Example:

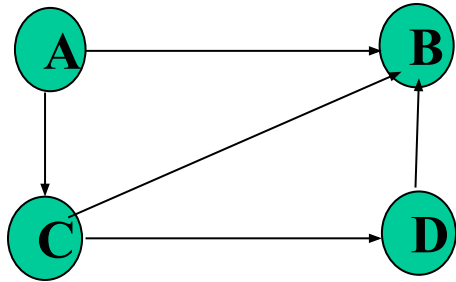


Figure 3: Directed Graph G

In graph G

Indeg(A)= 0 Outdeg(A)=2

Indeg(B)= 3 Outdeg(C)= 2

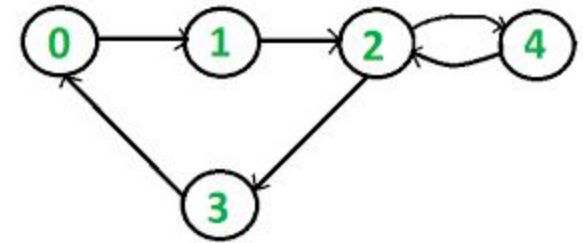
Indeg(C)= 1 Outdeg(B)= 0

Indeg(D)= 1 Outdeg(D)= 1

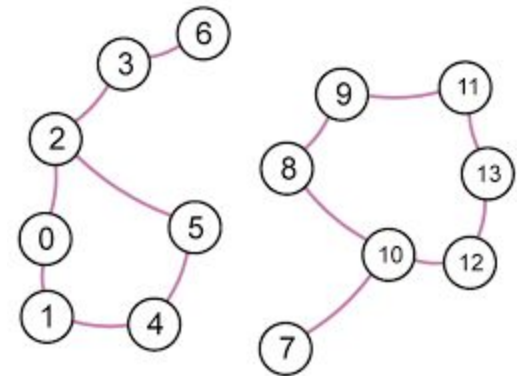
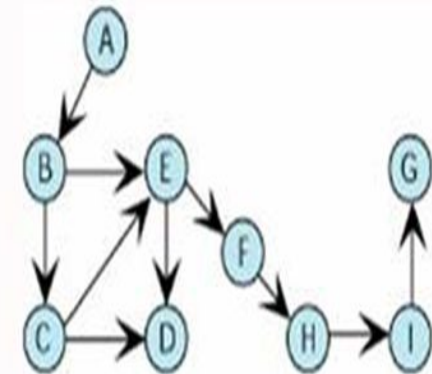
Note:

- A node u is called source if it has a positive outdegree and 0 indegree.
- A node u is called sink if it has a positive indegree and 0 outdegree.
- For a directed graph, a loop adds one to the indegree and one to the outdegree.
- For undirected graph, a loop adds two to the degree.

- **Strongly Connected:** A directed graph is strongly connected if there is a path from each vertex to other vertex
- **Weakly connected :** A directed graph is weakly connected if at least two vertices are not connected
- **Disjoint Graph:** if graph is not connected



Strongly Connected



Graph Representation

- Two popular computer representations of a graph. Both represent the vertex set and the edge set, but in different ways.

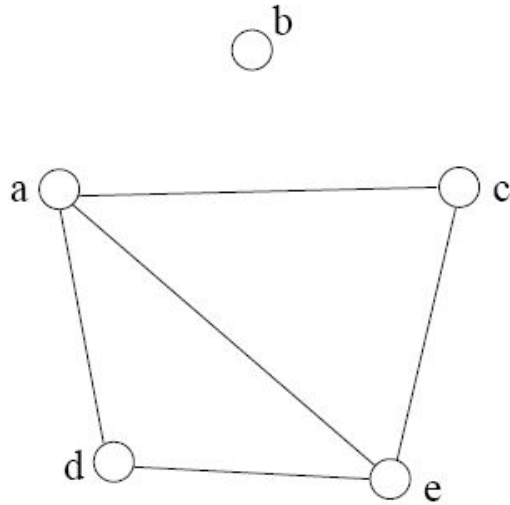
1. Adjacency Matrix

Use a 2D matrix to represent the graph

2. Adjacency List

Use a 1D array of linked lists

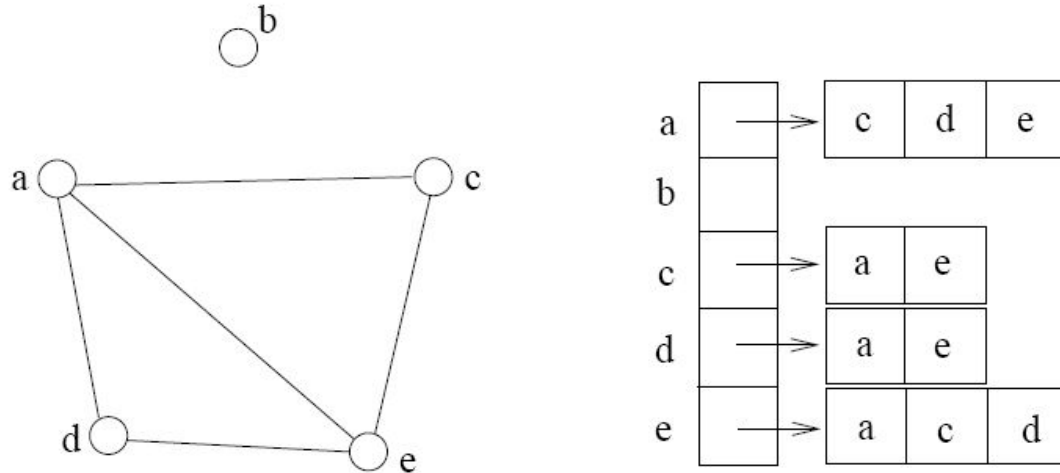
Adjacency Matrix



	a	b	c	d	e
a	0	0	1	1	1
b	0	0	0	0	0
c	1	0	0	0	1
d	1	0	0	0	1
e	1	0	1	1	0

- 2D array $A[0..n-1, 0..n-1]$, where n is the number of vertices in the graph
- Each row and column is indexed by the vertex id
 - e.g. $a=0, b=1, c=2, d=3, e=4$
- $A[i][j]=1$ if there is an edge connecting vertices i and j ; otherwise, $A[i][j]=0$

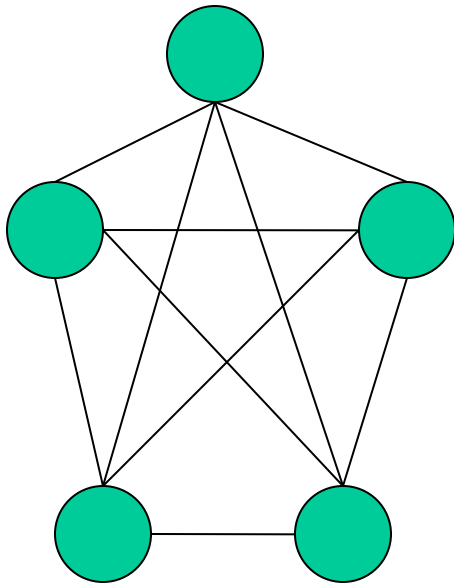
Adjacency List



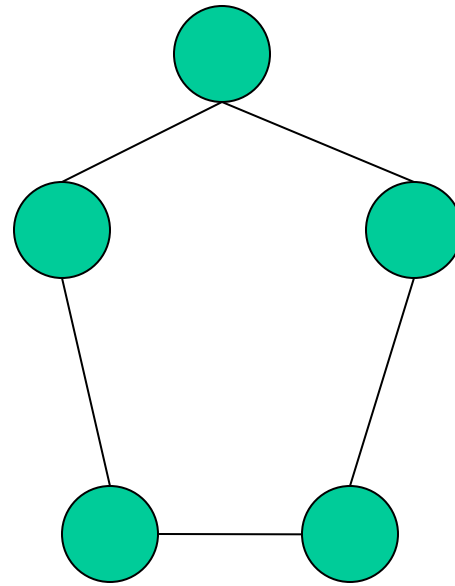
- If the graph is not dense, in other words, sparse, a better solution is an adjacency list
- The adjacency list is an array $A[0..n-1]$ of lists, where n is the number of vertices in the graph.
- Each array entry is indexed by the vertex id
- Each list $A[i]$ stores the ids of the vertices adjacent to vertex i

Which Representation Is Better?

- The adjacency-list is usually preferred if the graph is sparse (having few edges).
- The adjacency-matrix is preferred if the graph is dense (the number of edges is close to the maximal number of edges).



Dense Graph



Sparse Graph

Figure: Dense and Sparse Graph

END!!!